



Benchmarking RAG Systems for LLMs

A Real-World Resume Analysis

A real tool for Human Capital Hiring
Data Engineering & Performance
Built in .NET and Python

Who I Am



Mauro Gioberti

Software Engineer | Mentor

Experience

15+ years building backend systems

Real-world, production environments

- Backend & distributed systems
- Clean architecture & maintainable code
- Testing and automation mindset
- Mentoring and knowledge sharing



What Are We Benchmarking

System Behavior

Not model intelligence. We're testing how different frameworks handle the same RAG workflow.

Controlled Variables

Same dataset, prompts, and evaluation criteria across both implementations.

Language Stack Differences

Different frameworks and language ecosystems solving identical problems.

Core Focus

Consistency, grounding, and reliability. Not "which LLM is smarter."

Why This Is a Real-World Case

The Challenge

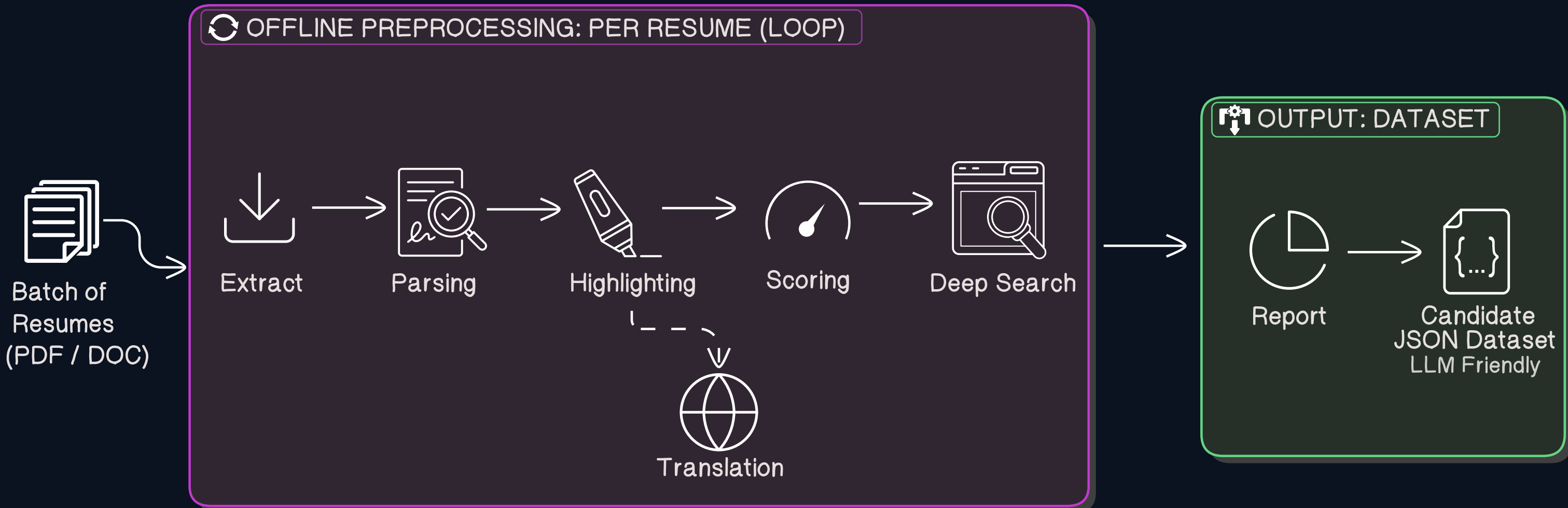
Real resume data is inherently messy and inconsistent, presenting challenges that clean demo datasets never reveal.

- CVs arrive in PDFs, DOCs, with wildly different layouts and languages
- Missing data and duplicated information are the norm
- Ambiguous work histories and noisy signals throughout
- This is authentic HR data, not a sanitized benchmark



From Raw CVs to Usable Data

Our preprocessing pipeline transforms unstructured resumes into queryable, structured candidate profiles through systematic extraction and enrichment.



CVs in the Wild

Unstructured. Inconsistent. Hard to compare.

Different Formats

PDF, DOCX, plain text, scanned images

Different Styles

Creative layouts, traditional formats, minimal designs

Different Signal Quality

Incomplete data, vague descriptions, inconsistent details

Preprocessing Output: Candidate Profile

Structured Dataset

Machine-readable candidate representation

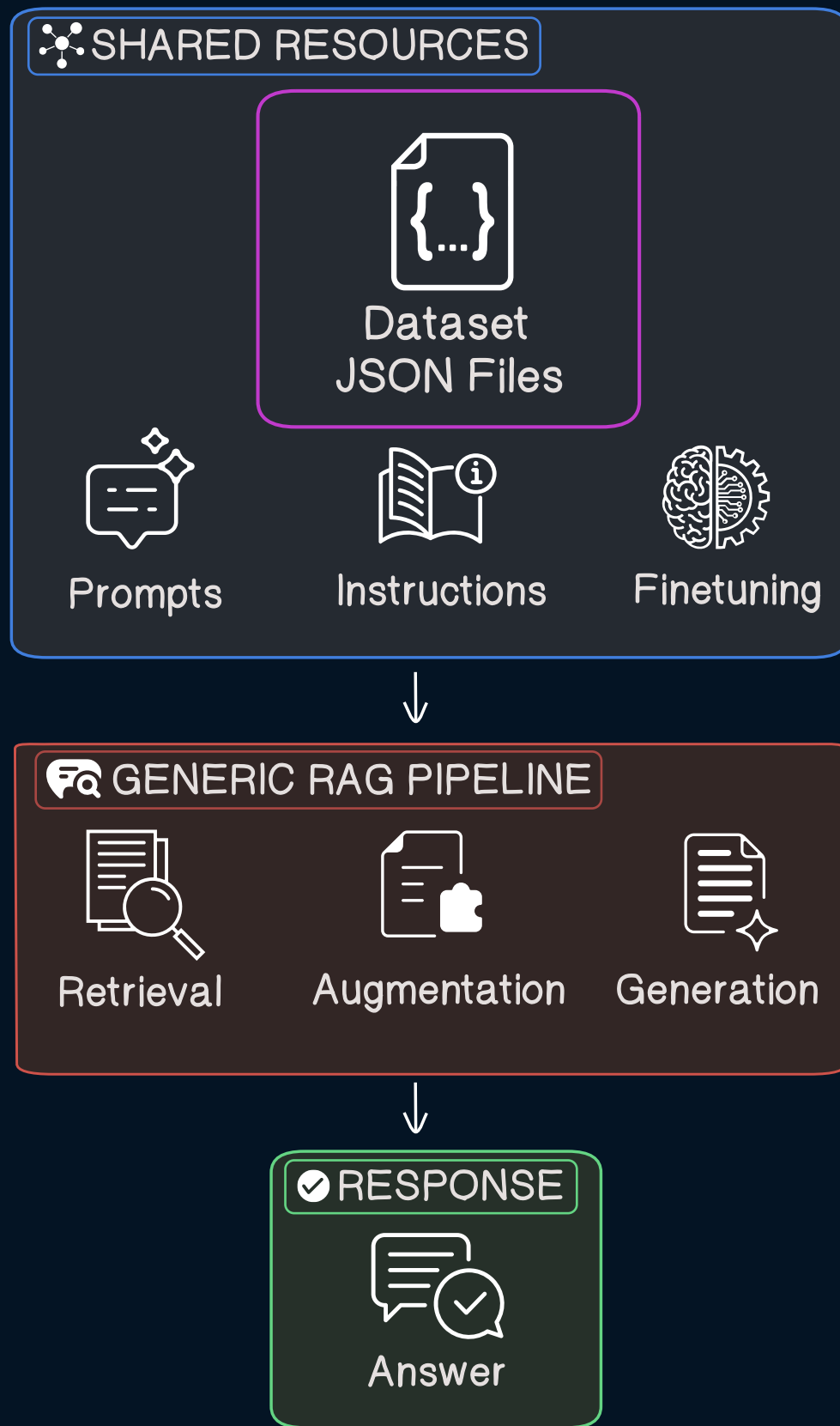
The preprocessing pipeline outputs normalized, machine-readable candidate profiles that enable fair comparison and deep querying.

- Normalized candidate profile
- Scoring and key signals
- Strengths and keyword coverage
- Clean, comparable JSON format

```
JSON Output Example

{
  "Profile": {
    "Title": "Senior Backend .NET Engineer / Tech Lead",
    "EnglishLevel": "C1",
    "Industry": "Fintech / Enterprise"
  },
  "TopSkills": [".NET C#", "Microservices", "Azure / AWS"],
  "Keywords": {
    "Found": ["Tech Lead", "Mentor", "..."],
    "Missing": ["Cloud", "DevOps", "..."]
  },
  "Scores": {
    "Overall": 92,
    "SeniorityMatch": 95,
    "ATS": 88
  }
}
```

Example of a single candidate entry in the dataset. (*dataset excerpt*)



Feeding the RAG System

Controlled inputs. Shared context. Comparable outputs.

A generic RAG flow powered by shared, immutable inputs. All systems consume the same dataset, prompts, and instructions before any implementation-specific behavior is introduced.

01

Shared Resources

A structured JSON dataset generated during preprocessing, combined with shared prompts, instructions, and finetuning configuration. These resources remain fixed to ensure a controlled baseline.

02

Generic RAG Pipeline

A framework-agnostic RAG flow composed of retrieval, augmentation, and generation. The pipeline is intentionally abstracted to isolate execution differences in later stages.

03

Answer

A final generated response produced under identical input conditions. This output becomes the basis for comparison across different implementations.

Introducing the Applications

Two chatbots, same goal

Chatbot #1: .NET



- Minimal API: ASP.NET Core
- Semantic Kernel
- Microsoft ecosystem

Chatbot #2: Python



- FastAPI
- LangChain
- Python ecosystem

Note: The .NET implementation has been updated to use [Microsoft.Extensions.AI](#) as the base abstraction. Semantic Kernel now plays a role as an orchestration layer.

Two Identical Implementations

Fair comparison by design



Same Dataset

Identical preprocessed JSON input



Same Prompts

Shared prompts & instructions



Same Questions

Identical queries and execution flow



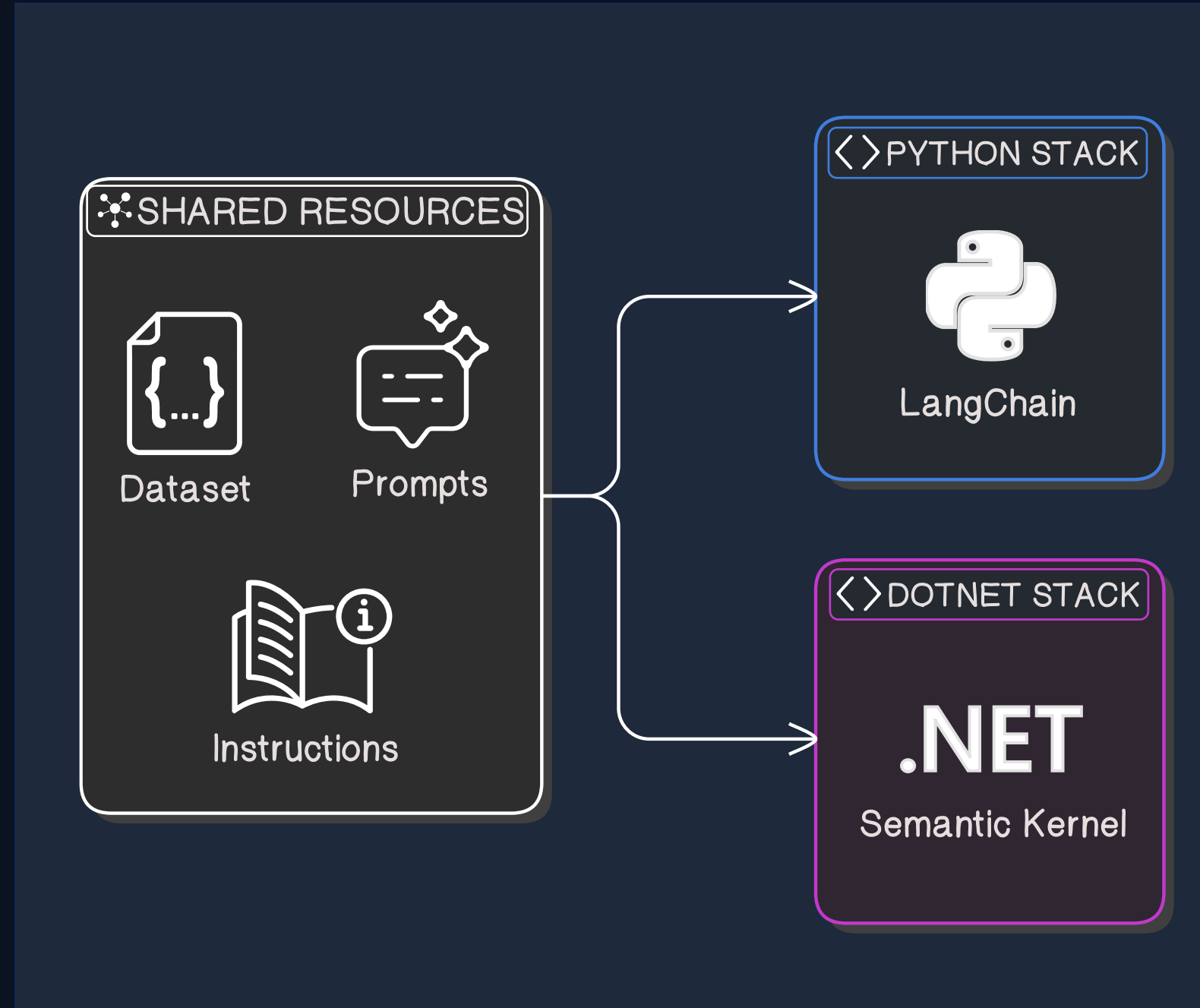
Same Evaluation Criteria

Consistent scoring and comparison rules

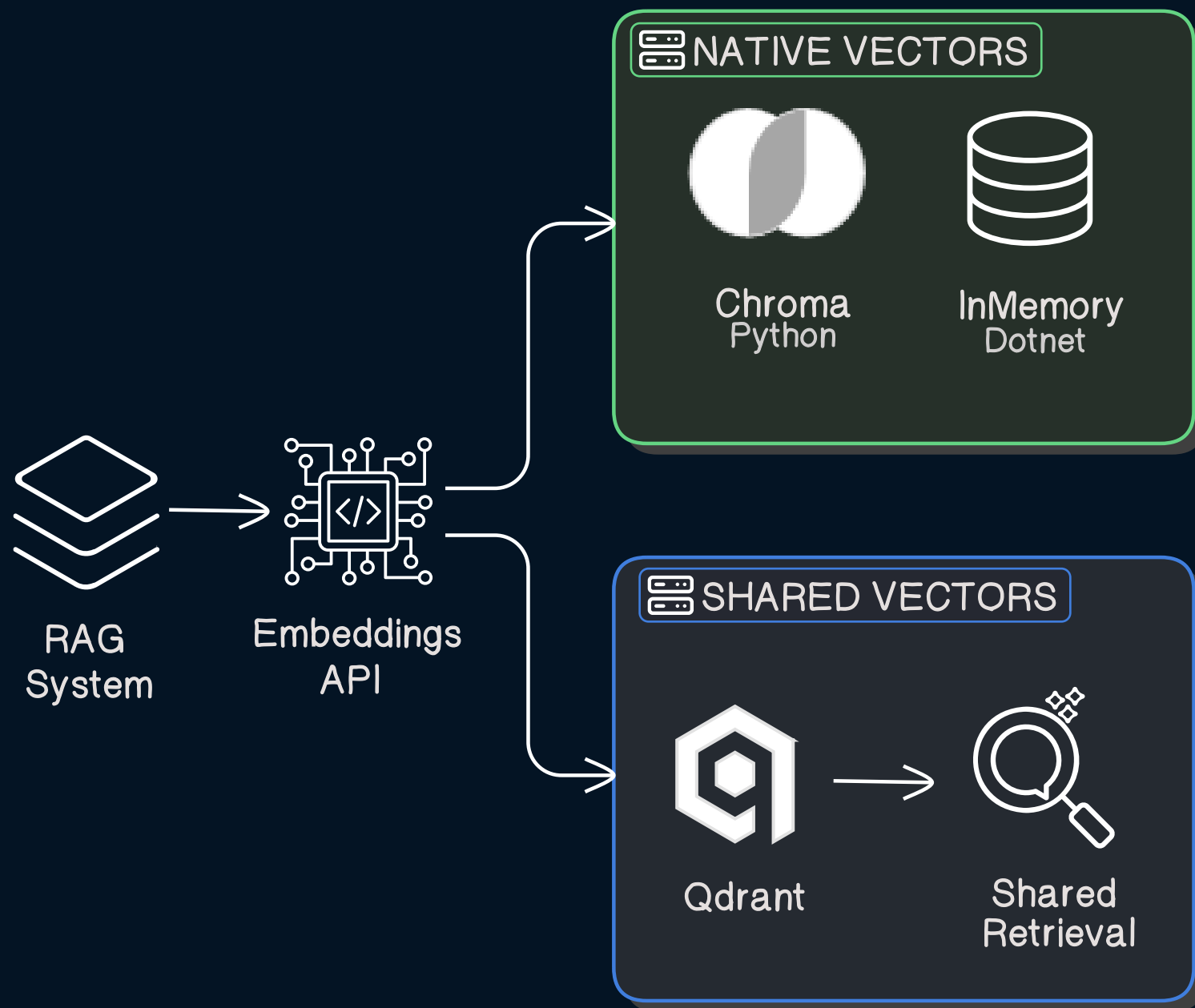


Different Stacks

Only the framework and language change



Only one thing changes: Language and framework stack. Same Input, different stacks.



Storage Variants

Retrieval strategies compared

We tested both native vector stores and external databases to measure retrieval quality and performance across identical RAG implementations.

Native Vector Stores

Python: Chroma for embedded vector storage

.NET: Custom in-memory vector store implementation

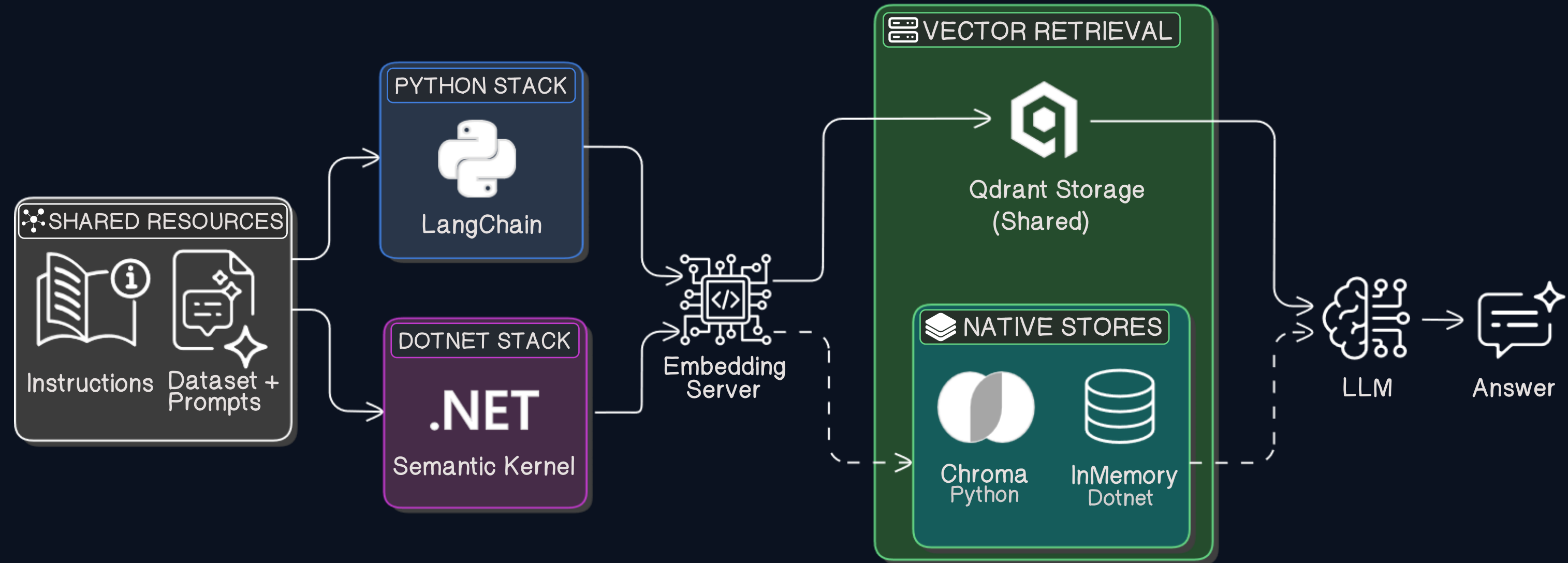
External Vector Store

Qdrant: Shared by both Python and .NET systems for consistent retrieval performance

* InMemory: Is a custom implementation (Dotnet SK.Plugins.Memory is still alpha)

RAG Execution Architecture

Full system overview





Demo

1

Same Dataset

Identical resume corpus and document preprocessing

2

Same UI

Backend selector toggles between .NET and Python

3

Same Question

Identical prompts and query formulation

Key observation: Both systems behave identically before benchmarking begins.



Benchmark Design

Configurations

- Python vs .NET
- Native vs Qdrant retrieval
- Same question set for all

Evaluation Method

- LLM-as-a-judge scoring
- 0–10 scale per response
- Blind comparison mode

Key Metrics

- Win/Tie outcome
- Win distribution
- Tie rate percentage
- Average score delta

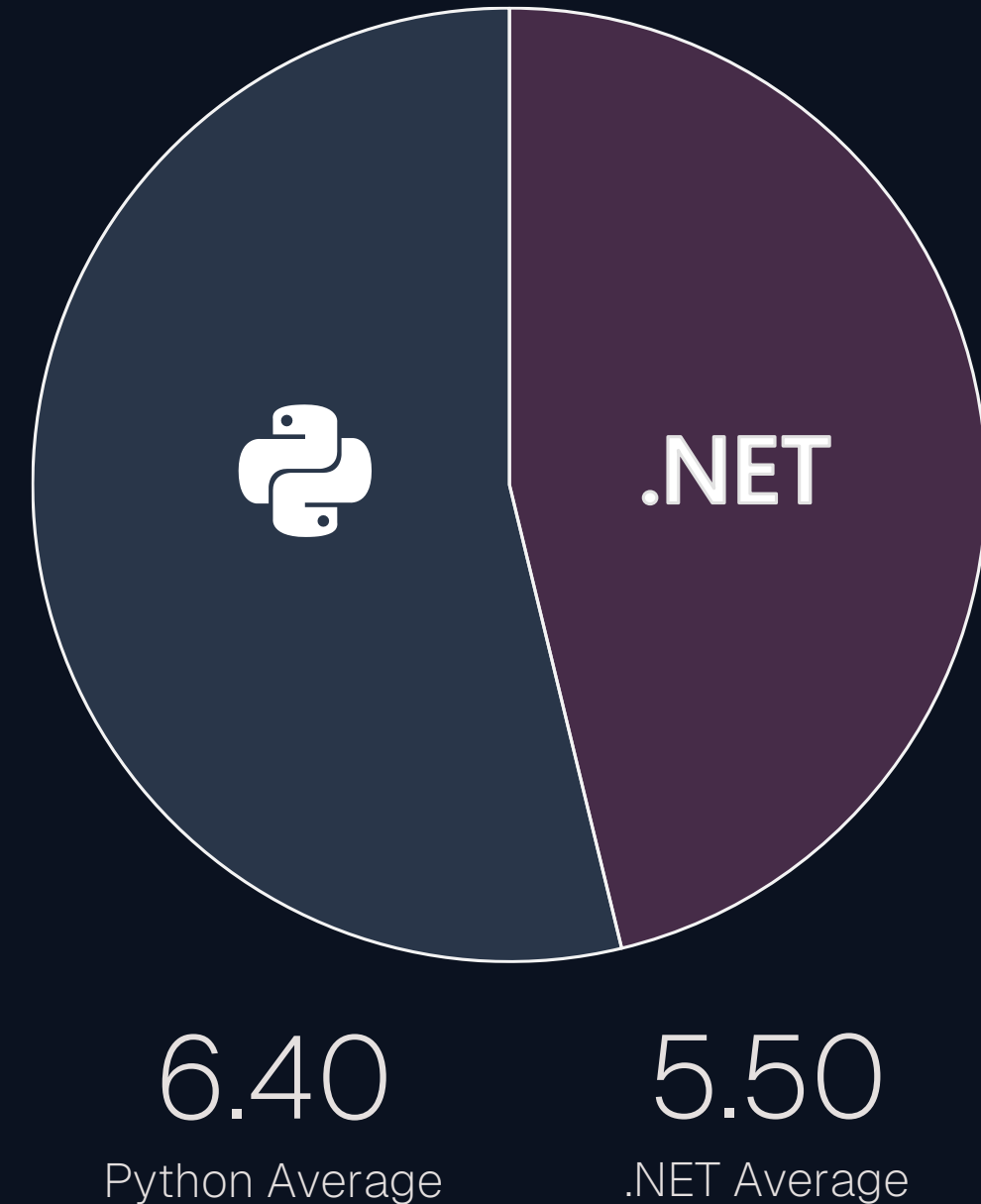
Only the stack and retrieval strategy change between runs.

Native Vector Storage Results

With native stores, framework-specific retrieval differences are more visible.

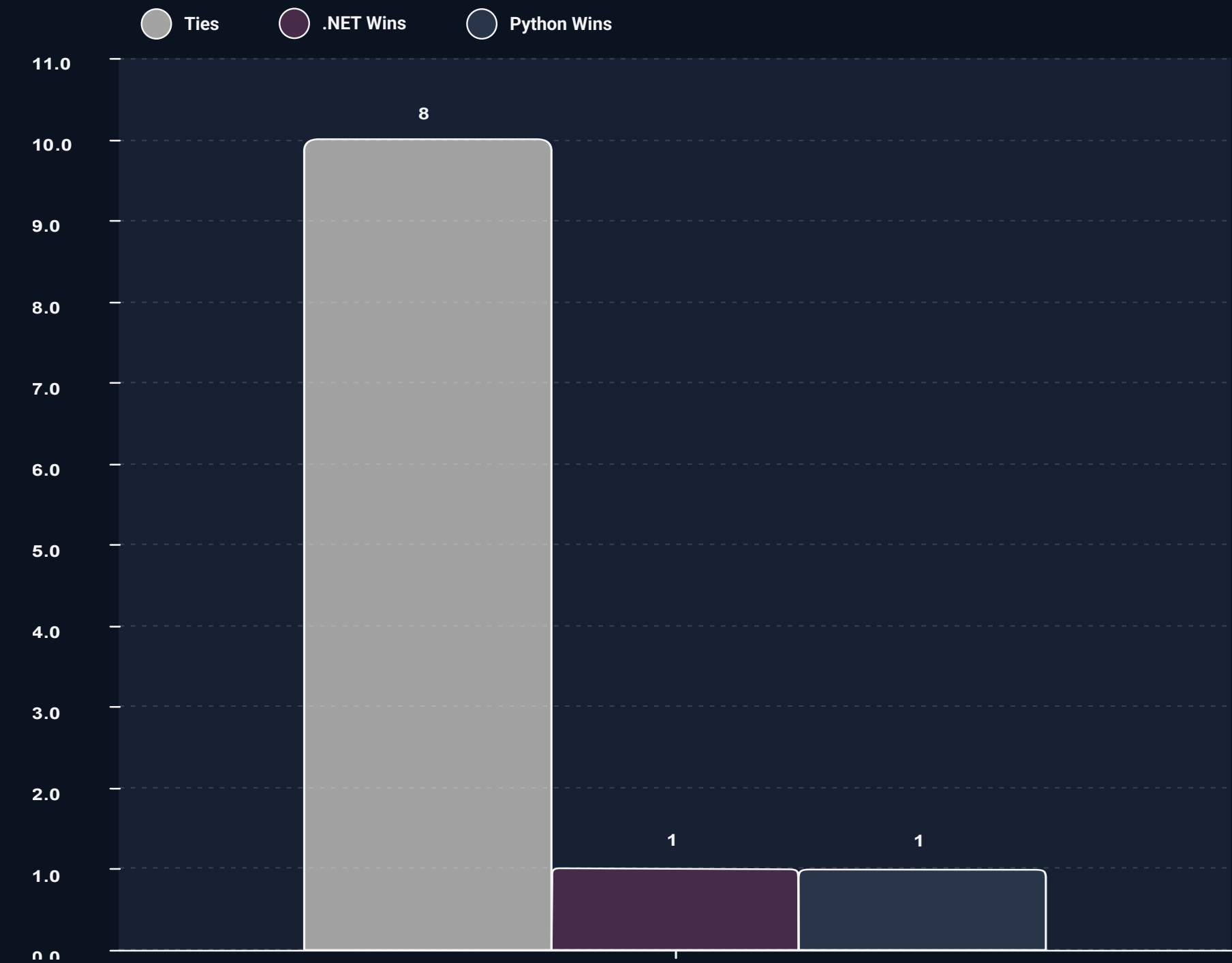


Distribution of Native Average Scores
(0-10) (score)

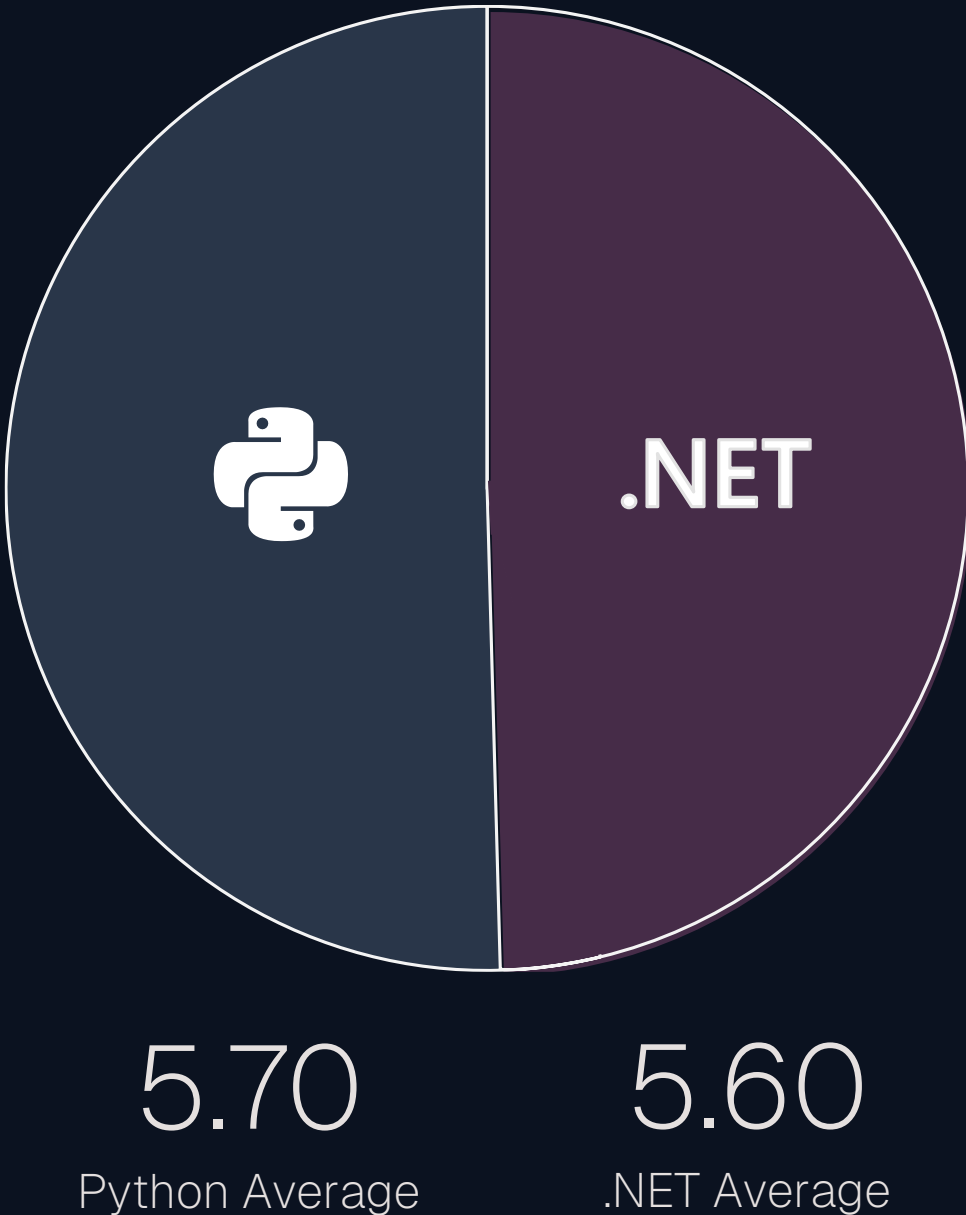


Qdrant Vector Storage Results

With shared vector stores, framework-specific retrieval differences become less visible.

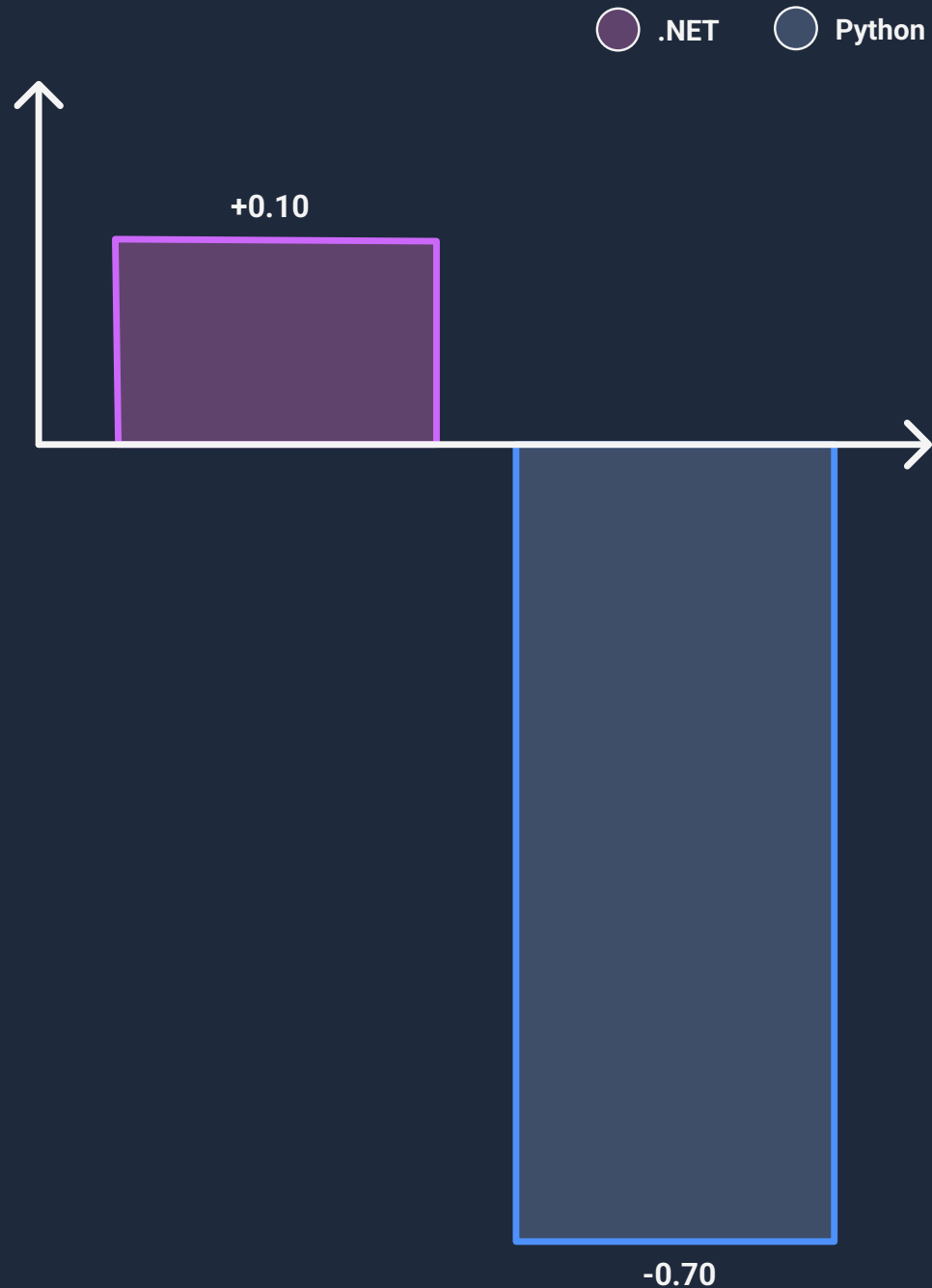


Distribution of Qdrant Average Scores (0-10) (score)



Delta Score

Score change with shared retrieval



Comparison Summary

Native Retrieval

- Framework differences visible
- Score variance higher
- Implementation details matter
- Tie rate lower

Qdrant Retrieval

- Tie rate increases significantly
- Average scores converge
- .NET improves with better retrieval
- Framework choice less impactful

Critical insight: As retrieval quality improves, framework differences disappear.

Benchmark Summary



STABILITY

Tie Rate: NATIVE 50% → QDRANT 80%

Comparisons Tied:

- 5/10 Native
- 8/10 Qdrant



SCORE

Gap: NATIVE 0.90 → QDRANT 0.10

Average Scores:

	NATIVE	QDRANT
.NET	5.50	5.60
Python	6.40	5.70



RETRIEVAL

Wins become Balanced

UNDER SHARED RETRIEVAL

Native Wins: .NET 2, Python 3

Qdrant Wins: .NET 1, Python 1

Key Learnings

Retrieval Dominates

Retrieval quality has far more impact than framework choice or language ecosystem.

External Databases Reduce Bias

Shared vector databases eliminate framework-specific retrieval quirks and inconsistencies.

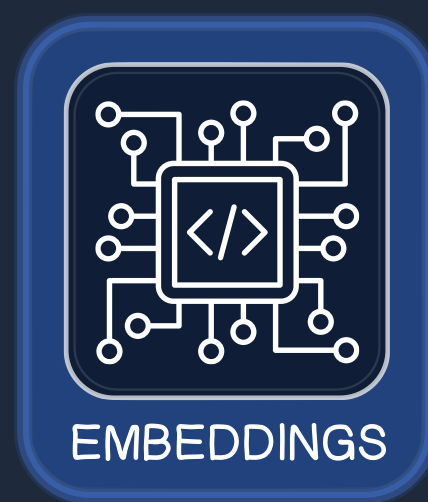
Higher Tie Rate Often Signals Stability

When systems tie frequently, it indicates both are behaving consistently and predictably.

Real Data Essential

Fair benchmarks require real-world data, not synthetic or curated test cases.

Frameworks don't win benchmarks, architectures & design do.



Final Takeaways

01

Preprocessing is Mandatory

Real-world data requires careful cleaning, normalization, and preparation before ingestion.

03

Parity is Achievable

Python and .NET reach equivalent performance with strong retrieval infrastructure.

02

Control Your Variables

Benchmarks must isolate the factors you're testing. Control inputs, prompts, and evaluation.

04

Infrastructure Matters Most

Your choice of vector database and retrieval strategy outweighs language choice.



Q&A Backup

- Clarifications

Technical details about benchmark methodology or implementation

- Extra Questions

Deep dives into specific configurations or unexpected results

- Edge Cases

Discussion of outliers, failure modes, or special scenarios

THANK YOU

Keep in Touch

Mauro Gioberti

.NET Software Engineer

maurogioberti.com

Github: [/maurogioberti](https://github.com/maurogioberti)

Linkedin [/maurogioberti](https://linkedin.com/in/maurogioberti)



TALK MATERIALS

Slides · Source Code · Docs · Feedback Form

Scan once. Access everything